



our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Rewardy

Dual-Entry Smart Account

SECURITY REVIEW

Date: 23 April 2026

CONTENTS

1. About Shieldify Security	3
2. Disclaimer	3
3. About Rewardy - Dual-Entry Smart Account	3
4. Risk classification	5
4.1 Impact	5
4.2 Likelihood	5
5. Security Review Summary	5
5.1 Protocol Summary	5
5.2 Scope	5
6. Findings Summary	6
7. Findings	6

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Rewardy - Dual-Entry Smart Account

Rewardy Wallet is a blockchain-based WEB3 entertainment wallet that allows users to earn points through community activities. Users may manage their assets easily and securely with no gas fees, and earn points for being active in 20+ global communities.

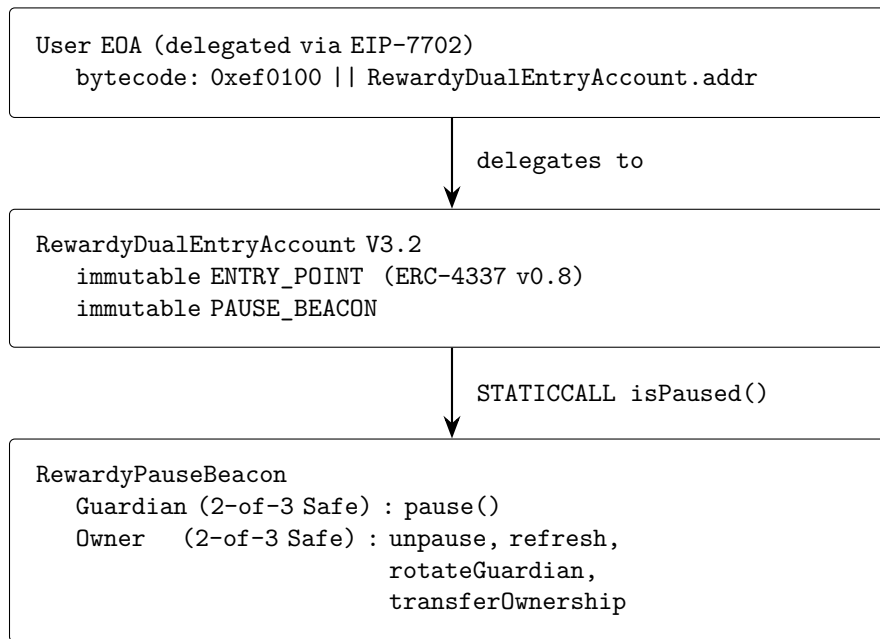
RewardyDualEntryAccount is a dual-entry smart account designed to be used as an **EIP-7702 delegation target** for EOAs, while remaining fully compatible with **ERC-4337 v0.8** bundler flows. V3.2 adds an **external scope-limited pause authority** via a separate immutable beacon contract, enabling emergency incident response without compromising the account's minimal-trust design. It is the account contract powering Rewardy Wallet, Korea's first native EIP-7702 wallet.

The system is composed of two contracts:

- **RewardyDualEntryAccountV3.2** — the delegation target.
 - **EIP-7702 path**: off-chain EIP-712 signed batched execution with optional ETH/ERC-20 fees.
 - **ERC-4337 path**: standard `validateUserOp()` + `execute` / `executeBatch` guarded by an immutable `EntryPoint`.
- **RewardyPauseBeacon** — the external pause authority, governed by a separated Guardian / Owner multisig topology.

Deployment Warning **RewardyDualEntryAccount** is an **EIP-7702 delegation target ONLY**. Do **not** send funds directly to the deployed contract address. All signature checks verify `recovered == address(this)`; in 7702 context, this resolves to the delegating EOA (whose key signs). On the standalone deployed instance, `address(this)` has no private key — no signature can ever pass, and any ETH or tokens sent there are **permanently unrecoverable**.

System Overview



Design Goals

- **7702-first:** Designed primarily as a delegation target for user EOAs, with 4337 support as a secondary path for account-abstracted flows.
- **Scope-limited pause:** An incident-response circuit breaker that gates only sponsored execution paths. Inbound transfers, signature verification, and direct-mode EOA transactions are never affected.
- **Separated governance:** Two-key topology. A hot **Guardian** multisig can only pause; a cold **Owner** multisig holds recovery and management authority. No single role can move or freeze user funds.
- **Minimal beacon surface:** The beacon exposes only the functions strictly necessary for its purpose — pause, unpause, refresh, rotateGuardian, transferOwnership. Recovery from a compromised guardian is achieved by the Owner sequencing rotateGuardian before unpause; no atomic bundled recovery path exists in the contract, by design, to keep the on-chain surface minimal and auditable.
- **Self-healing failure modes:** Pause auto-expires after 24h. Beacon destruction fails open.
- **Minimal trusted surface:** Both ENTRY_POINT and PAUSE_BEACON are immutable. No upgradability, no admin setter, no proxy, no selfdestruct, no DELEGATECALL.
- **Signature transparency:** EIP-712 typed struct array so wallets can render each call to the user before signing.
- **Forward-compatible storage:** `_gap[47]` reserved for future state additions without corrupting delegated-EOA storage.
- **dApp compatibility:** Raw-hash EIP-1271 preserves compatibility with Uniswap Permit2, SeaPort, Ox, CowSwap, SIWE, and WalletConnect. DEX aggregator patterns (linch, Ox, ParaSwap), including slippage buffer and refund flows, work out of the box.

[More docs](#)

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol’s overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to grieving attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 4 days, with a total of 64 hours dedicated to the audit by two researchers from the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying three Medium and four Low severity issues. They’re related to deviations from ERC-4337 account semantics, unsafe signature validation and replay risks, and missing execution and storage safeguards that open the door to gas grieving, stale operation reuse and unreachable modifier.

The Rewardy team has done a great job with their test suite and provided exceptional support, and promptly implemented all of the suggested recommendations from the Shieldify researchers.

5.1 Protocol Summary

Project Name	Rewardy – Dual-Entry Smart Account
Repository	rewardy-7702
Type of Project	EIP-7702, ERC-4337
Security Review Timeline	4 days
Review Commit Hash	c3be69df65ac6eaaed393ab7e30ab454cd245ae1
Fixes Review Commit Hash	1c3f970f6e1d6034bb3e17bd18c102478f093aa3

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
rwd.dual.entry.account.v3.sol	339
rwd.pause.beacon.sol	73
Total	412

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 3
- **Low** issues: 4
- **Info** issues: 2

ID	Title	Severity	Status
[M-01]	<code>validateUserOp()</code> Deviates From ERC-4337 Account Semantics	Medium	Fixed
[M-02]	Raw-Hash ERC-1271 Validation Enables Cross-Context Signature Replay	Medium	Fixed
[M-03]	ERC-4337 UserOperations Have No Account-Level Validity Window	Medium	Fixed
[L-01]	Unbounded Return Data Copy in <code>_executeBatchNoNonce()</code> Enables Gas Griefing	Low	Fixed
[L-02]	Missing ERC-7201 Namespaced Storage Violates EIP-7702 Requirement and Endangers Re-Delegation Safety	Low	Fixed
[L-03]	Stale ERC-4337 UserOperation Signatures Remain Valid Across EIP-7702 Re-delegation	Low	Fixed
[L-04]	<code>whenNotPaused</code> Modifier is Not Reachable	Low	Fixed
[I-01]	<code>validateUserOp()</code> Missing Explicit <code>initCode</code> Rejection	Info	Fixed
[I-02]	Unused Interface Declarations	Info	Fixed

7. Findings

[M-01] `validateUserOp()` Deviates From ERC-4337 Account Semantics

Severity

Medium Risk

Description

Rewardy markets the account as ERC-4337 compatible, but its `validateUserOp()` implementation deviates from the ERC-4337 account-validation spec in two concrete ways:

1. Non-signature errors return `SIG_VALIDATION_FAILED` instead of reverting.

The ERC-4337 spec states that the account “SHOULD return `SIG_VALIDATION_FAILED` (and not revert) on signature mismatch. **Any other error MUST revert.**” Rewardy returns `SIG_VALIDATION_FAILED` (`1`) for pause state and sender mismatch — neither of which is a signature error. These should revert.

The `userOp.sender != address(this)` check is itself redundant since the `EntryPoint` guarantees `userOp.sender == account_address` before calling `validateUserOp()`, so the return-vs-revert distinction has no practical impact for that branch. The pause check, however, can realistically trigger.

2. Signature failure path returns early without paying `_payPrefund()`.

In the eth-infinitism reference `BaseAccount`, the flow is:

```
// Reference pattern [eth-infinitism BaseAccount]
validationData = _validateSignature(userOp, userOpHash);
_validateNonce[userOp.nonce];
_payPrefund[missingAccountFunds]; // always called, even when sigFailed
```

Prefund is paid **regardless** of the signature outcome. In Rewardy’s implementation, the signature-mismatch branch (lines 320-321) returns `SIG_VALIDATION_FAILED` early and skips `_payPrefund()`. Per the reference pattern, even when the signature fails and the function returns `1`, the account should still pay the prefund to keep the `EntryPoint`’s deposit accounting consistent and bundler gas estimation reliable.

This is distinct from the non-signature error paths (pause, sender mismatch). Once those are fixed to revert per point 1, the prefund question is moot on those paths since a revert rolls back all state. The prefund-skipping problem only persists on the signature failure path, which correctly returns `1` but incorrectly exits before paying.

Note: the `pop(call(...))` pattern inside `_payPrefund()` that ignores the transfer return value is **standard and correct**. The eth-infinitism reference does the same and comments “ignore failure [its `EntryPoint`’s job to verify, not account.]”. The issue is not the `pop(call(...))` pattern; it is that `_payPrefund()` is never reached on the signature failure path.

3. README misrepresents spec compliance.

README line 372 states: “All failures return `SIG_VALIDATION_FAILED` (`= 1`) per the ERC-4337 spec; bundler-breaking reverts are avoided.” This is the opposite of what the spec says non-signature errors **must** revert.

Location of Affected Code

`validateUserOp()` — non-signature errors return `1` instead of reverting, and signature failure returns early before prefund:

File: [rwd.dual.entry.account.v3.sol#L269-L293](#)

```
// rwd.dual.entry.account.v3.sol:299-326
function validateUserOp(
    PackedUserOperation calldata userOp,
    bytes32 userOpHash,
    uint256 missingAccountFunds
) external onlyEP returns (uint256 validationData) {
    if [_isPaused()] {
        return SIG_VALIDATION_FAILED; // @audit should revert — not a
        signature error
    }

    if (userOp.sender != address(this)) {
        return SIG_VALIDATION_FAILED; // @audit should revert — not a
        signature error
    }

    (address recovered, ECDSA.RecoverError err, ) = ECDSA.tryRecover(
        userOpHash,
        userOp.signature
    );

    if (err != ECDSA.RecoverError.NoError || recovered != address(this))
    {
        return SIG_VALIDATION_FAILED; // @audit correct return value,
        but skips _payPrefund
    }

    _payPrefund(missingAccountFunds); // @audit only reached on success
    path
    return SIG_VALIDATION_SUCCESS;
}
```

`_payPrefund()` — never reached on signature failure path:

File: [rwd.dual.entry.account.v3.sol#L296-L302](#)

```
// rwd.dual.entry.account.v3.sol:329-335
function _payPrefund(uint256 missingAccountFunds) internal {
    if (missingAccountFunds > 0) {
        assembly {
            pop(call(gas(), caller(), missingAccountFunds, 0, 0, 0, 0))
        }
    }
}
```

Impact

- **Spec non-compliance:** Returning `1` for non-signature errors violates the ERC-4337 “any other error MUST revert” requirement.
- **Prefund skipped on signature failure:** The signature mismatch path returns `1` (correct) but

skips `_payPrefund()` (incorrect per reference pattern), causing inconsistent gas estimation and deposit accounting.

- **README misrepresentation:** Integrators relying on the documented claim of “all failures return 1 per spec” will build against incorrect assumptions.

Proof of Concept

Scenario 1 — Pause returns 1 instead of reverting: 1. The guardian pauses the system via the pause beacon. 2. A user submits a valid `UserOperation` through a bundler. 3. The bundler simulates `validateUserOp()` — `_isPaused()` returns `true`. 4. Instead of reverting (spec-required for non-signature errors), the function returns `SIG_VALIDATION_FAILED` and exits early. 5. A strict or standards-following bundler can flag or reject the account as non-canonical.

Scenario 2 — Signature failure skips prefund: 1. A user submits a `UserOperation` with an invalid signature. 2. `validateUserOp()` reaches the ECDSA check, signature fails, and the function returns `SIG_VALIDATION_FAILED` at line 321. 3. `_payPrefund()` at line 324 is never executed. 4. In the reference `BaseAccount` pattern, prefund would still be paid here — ensuring the EntryPoint’s deposit accounting and gas estimation remain consistent even on failed validations.

Recommendation

1. **Revert on non-signature errors** — pause and sender mismatch should revert, not return 1. Once these revert, the prefund is automatically moot on those paths (reverts roll back all state).
2. **Always pay prefund on the signature failure path** — restructure the function so that `_payPrefund()` executes on both signature success and signature failure, matching the reference `BaseAccount` pattern. The signature check should set a flag rather than return early.
3. **Update the README** — remove the claim that “all failures return 1 per spec.”

A minimally corrected shape:

```
function validateUserOp(
    PackedUserOperation calldata userOp,
    bytes32 userOpHash,
    uint256 missingAccountFunds
) external onlyEP returns (uint256 validationData) {
    // Non-signature errors revert per spec.
    // Prefund is moot here — revert rolls back all state.
    if [_isPaused()] revert SystemPaused();
    if (userOp.sender != address(this)) revert InvalidSender();

    // Signature check sets a flag instead of early-returning.
    bool sigFailed;
    {
        (address recovered, ECDSA.RecoverError err, ) = ECDSA.tryRecover(
            userOpHash,
            userOp.signature
        );
        sigFailed = (
```



```

        err != ECDSA.RecoverError.NoError || recovered != address(
            this)
    };
}

// Always pay prefund — even on signature failure — matching
// reference BaseAccount. EntryPoint validates deposit balance;
// account ignores call result per standard pattern.
if (missingAccountFunds > 0) {
    assembly {
        pop(call(gas(), caller(), missingAccountFunds, 0, 0, 0, 0))
    }
}

return sigFailed ? SIG_VALIDATION_FAILED : SIG_VALIDATION_SUCCESS;
}

```

Team Response

Fixed.

[M-02] Raw-Hash ERC-1271 Validation Enables Cross-Context Signature Replay

Severity

Medium Risk

Description

The ERC-1271 verifier accepts an externally supplied raw hash and verifies it directly with `ECDSA.tryRecover(hash, signature)`.

Because the account does not enforce EIP-712 domain wrapping at the ERC-1271 boundary, signature safety depends entirely on whatever digest the external caller provides. If that digest is not robustly domain-separated (account + chain), signatures can be replayed/transplanted across contexts (including cross-chain contexts that share the same account address).

Location of Affected Code

File: [rwd.dual.entry.account.v3.sol#L495-L505](#)

```

function isValidSignature(
    bytes32 hash,
    bytes calldata signature
) external view returns (bytes4) {
    (address recovered, ECDSA.RecoverError err, ) = ECDSA.tryRecover(
        hash,
        signature
    );
    if (err != ECDSA.RecoverError.NoError) return EIP1271_FAILED;
}

```

```
return recovered == address[this] ? EIP1271_SUCCESS : EIP1271_FAILED;
}
```

Impact

A phished or attacker-influenced signature over a raw digest can be accepted by this wallet as valid ERC-1271 authorization for downstream protocols (for example, permit/order/intent systems) that rely on `isValidSignature()`.

Depending on the downstream integration, this can lead to unauthorised approvals, fills, or transfers, and therefore user fund loss.

Proof of Concept

1. Attacker prepares a digest `H` that corresponds to a valuable downstream action.
2. Victim is induced to sign `H` (for example, via opaque/raw signing prompt).
3. Attacker submits `(H, sig)` to a downstream protocol that validates via `isValidSignature(H, sig)`.
4. The wallet returns ERC-1271 success (`0x1626ba7e`) because it verifies the raw digest directly.
5. The downstream protocol accepts the signature and executes the attacker-benefiting action.

Replay-focused negative tests that should currently fail under a hardened design: - Reuse the same signature against a different account context. - Reuse the same signature on a different chain. - Reuse the same signature after the signer/authority transition.

Recommendation

Use EIP-712 typed-data validation inside ERC-1271 so signatures are domain-separated by `chainId` and `verifyingContract`.

A code-based mitigation aligned with the curiousapple disclosure is to wrap the incoming `hash` in a typed struct that explicitly includes account and chain context, then recover against the EIP-712 digest:

```
bytes32 private constant ERC1271_TYPEHASH =
    keccak256("ERC1271Message(bytes32 hash,address account,uint256
        chainId)");

function isValidSignature(
    bytes32 hash,
    bytes calldata signature
) external view returns (bytes4) {
    bytes32 structHash = keccak256(
        abi.encode(ERC1271_TYPEHASH, hash, address(this), block.chainid)
    );

    // In this codebase, `_hashTypedData` already uses a domain separator
    // that
    // includes chainId and verifyingContract.
    bytes32 digest = _hashTypedData(structHash);
```

```

    (address recovered, ECDSA.RecoverError err, ) = ECDSA.tryRecover(
        digest,
        signature
    );

    if (err != ECDSA.RecoverError.NoError) return EIP1271_FAILED;
    return recovered == address(this) ? EIP1271_SUCCESS : EIP1271_FAILED;
}

```

Team Response

Fixed.

[M-03] ERC-4337 UserOperations Have No Account-Level Validity Window

Severity

Medium Risk

Description

The `validateUserOp()` returns raw `SIG_VALIDATION_SUCCESS` / `SIG_VALIDATION_FAILED` constants instead of returning fully packed ERC-4337 `validationData` with a validity window.

ERC-4337 `validationData` is a packed `uint256`:

- low 160 bits: aggregator / authorizer (`0` for self-validation, `1` for signature failure)
- next 48 bits: `validUntil`
- next 48 bits: `validAfter`

The account's success path returns `SIG_VALIDATION_SUCCESS`, which is `0`. The EntryPoint interprets this as:

- aggregator / authorizer = `address(0)`
- `validUntil = 0`, meaning no expiry
- `validAfter = 0`, meaning valid immediately

As a result, a signed ERC-4337 UserOperation is valid from the account's perspective indefinitely, until its EntryPoint nonce is consumed.

Location of Affected Code

File: [rwd.dual.entry.account.v3.sol#L56-L57](#)

```

uint256 internal constant SIG_VALIDATION_SUCCESS = 0;
uint256 internal constant SIG_VALIDATION_FAILED = 1;

```

File: [rwd.dual.entry.account.v3.sol#L269-L293](#)

```

function validateUserOp(
    PackedUserOperation calldata userOp,
    bytes32 userOpHash,
    uint256 missingAccountFunds
) external onlyEP returns (uint256 validationData) {
    if [_isPaused()] {
        return SIG_VALIDATION_FAILED;
    }

    if (userOp.sender != address[this]) {
        return SIG_VALIDATION_FAILED;
    }

    (address recovered, ECDSA.RecoverError err, ) = ECDSA.tryRecover(
        userOpHash,
        userOp.signature
    );

    if (err != ECDSA.RecoverError.NoError || recovered != address[this]) {
        return SIG_VALIDATION_FAILED;
    }

    _payPrefund(missingAccountFunds);
    return SIG_VALIDATION_SUCCESS;
}

```

Impact

A bundler or relayer holding a signed but unsubmitted UserOperation can submit it at a later time as long as the EntryPoint nonce remains unused and the account is not paused.

The stale UserOperation may execute after the user's intended window has passed. This is most relevant for time-sensitive calls such as swaps, approvals, or external protocol interactions that do not enforce their own deadline or slippage bounds.

The impact is limited because the relayer cannot modify the signed calldata and the UserOperation cannot be replayed after its nonce is consumed. This is therefore a stale-intent / liveness issue rather than an authorization bypass.

Proof of Concept

1. A user signs a valid ERC-4337 UserOperation for the account.
2. The UserOperation targets a time-sensitive action, for example, a swap or approval.
3. A bundler or relayer receives the signed UserOperation but does not submit it immediately.
4. The EntryPoint nonce remains unconsumed.
5. Hours or days later, the bundler submits the same UserOperation.
6. `validateUserOp()` returns `0`.
7. The EntryPoint decodes `0` as a valid self-validated operation with no `validUntil` and no `validAfter`, so the operation remains acceptable.

Recommendation

Add an explicit validity window to the ERC-4337 signature format and return packed validation data instead of raw success on the valid path.

The validity fields must be signed. Do not append an unsigned `validUntil` to `userOp.signature`.

Use a fixed-length signature payload, for example:

```
userOp.signature = 65-byte ECDSA signature || 6-byte validUntil || 6-byte validAfter
```

Finally, return ERC-4337-compatible packed validation data:

```
function _packValidationData(  
    bool sigFailed,  
    uint48 validUntil,  
    uint48 validAfter  
) internal pure returns (uint256) {  
    return  
        (sigFailed ? SIG_VALIDATION_FAILED : SIG_VALIDATION_SUCCESS) |  
        (uint256(validUntil) << 160) |  
        (uint256(validAfter) << 208);  
}
```

The validation flow should be:

```
bytes calldata validationSig = userOp.signature;  
  
if (validationSig.length != 77) {  
    _payPrefund(missingAccountFunds);  
    return _packValidationData(true, 0, 0);  
}  
  
bytes calldata sig = validationSig[:65];  
uint48 validUntil;  
uint48 validAfter;  
  
assembly {  
    validUntil := shr(208, calldataload(add(validationSig.offset, 65)))  
    validAfter := shr(208, calldataload(add(validationSig.offset, 71)))  
}  
  
bool invalidWindow = validUntil != 0 && validAfter > validUntil;  
  
// signature verification  
  
_payPrefund(missingAccountFunds);  
  
return _packValidationData(sigFailed, validUntil, validAfter);
```


Team Response

Fixed.

[L-01] Unbounded Return Data Copy in `_executeBatchNoNonce()` Enables Gas Griefing

Severity

Low Risk

Description

The `_executeBatchNoNonce()` uses Solidity's `(bool ok, bytes memory ret) = target.call{...}(...)` pattern, which copies the full `returndata_size()` into memory before branching on `ok`. On the revert path, `MAX_REVERT_DATA` limits how much data is bubbled, but the copy has already occurred. On the success path, no return data cap exists at all. A callee that returns an oversized payload forces excess memory expansion and copy costs on the caller.

Location of Affected Code

File: [rwd.dual.entry.account.v3.sol#L408-L420](#)

```
function _executeBatchNoNonce(
    InternalCall[] memory calls,
    uint256 nonceForEvent,
    bytes32 callsStructHash
) internal {
    // code
    (bool ok, bytes memory ret) = calls[i].to.call{
        value: calls[i].value
    }(calls[i].data);

    if (!ok) {
        if (ret.length > MAX_REVERT_DATA) revert CallReverted();
        assembly {
            revert(add(ret, 0x20), mload(ret))
        }
    }
    emit CallExecuted(calls[i].to, calls[i].value, calls[i].data);
    // code
}
```

Impact

An attacker-controlled callee can inflate gas usage for relayed or sponsored executions by returning oversized data on success. In a batch, this can cause the entire transaction to run out of gas, preventing legitimate calls in the same batch from executing.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {Test} from "forge-std/Test.sol";
import {RewardyDualEntryAccountV3} from "src/rwd.dual.entry.account.v3.sol";

contract NoPauseBeacon {
    function isPaused() external pure returns (bool) {
        return false;
    }
}

contract BenignReturnTarget {
    fallback() external payable {
        assembly { return(0x00, 0x00) }
    }
}

contract OversizedReturnTarget {
    uint256 internal constant RETURN_SIZE = 0x40000; // 256 KB

    fallback() external payable {
        assembly { return(0x00, RETURN_SIZE) }
    }
}

contract MockEntryPointGasHarness {
    function getNonce(address, uint192) external pure returns (uint256) {
        return 0;
    }

    function measureExecuteGas(
        RewardyDualEntryAccountV3 account,
        address target
    ) external returns (bool success, uint256 gasUsed) {
        uint256 gasBefore = gasleft();
        success = _callExecute(account, target, gasleft());
        gasUsed = gasBefore - gasleft();
    }

    function executeWithGasLimit(
        RewardyDualEntryAccountV3 account,
        address target,
        uint256 gasLimit
    ) external returns (bool success) {
        success = _callExecute(account, target, gasLimit);
    }
}
```

```

function _callExecute(
    RewardyDualEntryAccountV3 account,
    address target,
    uint256 gasLimit
) internal returns (bool success) {
    [success, ] = address(account).call{gas: gasLimit}(
        abi.encodeWithSelector(
            RewardyDualEntryAccountV3.execute.selector,
            target, 0, bytes("")
        )
    );
}

contract ReturnDataGasBombTest is Test {
    MockEntryPointGasHarness internal entryPoint;
    NoPauseBeacon internal beacon;
    RewardyDualEntryAccountV3 internal account;

    function setUp() external {
        entryPoint = new MockEntryPointGasHarness();
        beacon = new NoPauseBeacon();
        account = new RewardyDualEntryAccountV3(
            address(entryPoint), address(beacon)
        );
    }

    function test_fixedGasBudgetFailsOnlyForOversizedSuccessReturndata()
        external {
        BenignReturnTarget benignTarget = new BenignReturnTarget();
        OversizedReturnTarget oversizedTarget = new OversizedReturnTarget
            ();

        [bool benignOk, uint256 benignGas] =
            entryPoint.measureExecuteGas(account, address(benignTarget));
        [bool oversizedOk, uint256 oversizedGas] =
            entryPoint.measureExecuteGas(account, address(oversizedTarget
                ));

        assertTrue(benignOk, "benign target should execute");
        assertTrue(oversizedOk, "oversized return target should execute")
            ;
        assertGt(
            oversizedGas, benignGas + 30_000,
            "oversized return data should materially increase gas"
        );

        uint256 relayerBudget = benignGas + [(oversizedGas - benignGas) /
            2];

        bool benignFits =

```

```

        entryPoint.executeWithGasLimit(account, address(benignTarget)
            , relayerBudget);
    bool oversizedFits =
        entryPoint.executeWithGasLimit(account, address(
            oversizedTarget), relayerBudget);

    emit log_named_uint("Benign gas", benignGas);
    emit log_named_uint("Oversized gas", oversizedGas);
    emit log_named_uint("Shared relayer budget", relayerBudget);

    assertTrue(benignFits, "benign call should fit the relayer budget
        ");
    assertFalse(oversizedFits, "oversized success return data should
        exhaust the same budget");
}
}

```

```
forge test --match-path test/GasBomb.t.sol -vv
```

```

[PASS] test_fixedGasBudgetFailsOnlyForOversizedSuccessReturndata()
  Benign gas: 58701
  Oversized gas: 366604
  Shared relayer budget: 212652

```

A 256 KB success return inflates gas by ~308K compared to a zero-length return — enough to OOG a transaction budgeted for normal execution.

Recommendation

Replace the Solidity-level low-level call with an assembly call that avoids copying returndata into memory by default:

```

function _executeBatchNoNonce(
    InternalCall[] memory calls,
    uint256 nonceForEvent,
    bytes32 callsStructHash
) internal {
    // code
    for (uint256 i = 0; i < len; i++) {
        if (calls[i].to == address(this)) revert SelfCallForbidden();
        if (calls[i].to == ep) revert EntryPointCallForbidden();

        (bool ok, bytes memory ret) = calls[i].to.call{
            value: calls[i].value
        }(calls[i].data);

        if (!ok) {
            // Cap revert data size to prevent gas-bomb attacks from
            // malicious targets returning huge revert payloads.
            if (ret.length > MAX_REVERT_DATA) revert CallReverted();
        }
    }
}

```

```

-         assembly {
-             revert(add(ret, 0x20), mload(ret))
-         }
-     }
+     bool ok;
+     bytes memory data = calls[i].data;
+     address to = calls[i].to;
+     uint256 val = calls[i].value;
+     assembly {
+         ok := call(gas(), to, val, add(data, 0x20), mload(data), 0, 0)
+     }
+
+     if (!ok) {
+         uint256 rdz;
+         assembly { rdz := returndatasize() }
+         if (rdz > MAX_REVERT_DATA) revert CallReverted();
+         assembly {
+             let ptr := mload(0x40)
+             returndatacopy(ptr, 0, rdz)
+             revert(ptr, rdz)
+         }
+     }
+     emit CallExecuted(calls[i].to, calls[i].value, calls[i].data);
+ }
+ emit BatchExecuted(nonceForEvent, len, callsStructHash);
+ }

```

Team Response

Fixed.

[L-02] Missing ERC-7201 Namespaced Storage Violates EIP-7702 Requirement and Endangers Re-Delegation Safety

Severity

Low Risk

Description

The [EIP-7702 storage-management section](#) explicitly recommends storage isolation patterns such as [ERC-7201](#) for safe delegate-to-delegate migration:

“It’s important for these contracts to use storage in a way that avoids accidental collisions among them. For example, using ERC-7201, a contract may root its storage layout at a slot dependent on a unique identifier.”

The Rewardy delegation target uses plain sequential storage slots (`nonce` at slot 0, `_locked` at slot 1, `nonce4337` at slot 2, `__gap` at slots 3–49) instead of ERC-7201 namespaced storage. While the `__gap` array provides some forward-compatibility for adding new state variables within the same

contract family, it does NOT protect against storage collisions during re-delegation to a structurally different contract.

EIP-7702 further warns that if migration safety is uncertain, storage must be explicitly handled:

“If there is any doubt, it is recommended to first clear all account storage...”

This means that when a user redelegates from V3 to a future V4, the V4 contract reads V3's stale storage as its own. If V4 has a different slot layout, critical state variables like `nonce` (replay protection), `_locked` (reentrancy guard), and `nonce4337` (event counter) will be misinterpreted, potentially leading to signature replay, reentrancy guard bypass, or state corruption.

Location of Affected Code

File: [rwd.dual.entry.account.v3.sol#L92-L102](#)

```
uint256 public nonce;           // slot 0

bool private _locked;          // slot 1

/// @notice Event-only counter for the 4337 path. NOT used for replay
///         protection. Replay protection is handled entirely by the
///         EntryPoint's own nonce system; 'userOp.nonce' is included in
///         'userOpHash' which binds the signature.
uint256 public nonce4337;       // slot 2

uint256[47] private __gap;      // slots 3-49
```

The storage layout uses bare sequential slots with no ERC-7201 namespace derivation. Compare with the spec-compliant approach:

```
// ERC-7201 compliant layout (not currently implemented)
/// @custom:storage-location erc7201:rewardy.dualentry.v3
struct AccountStorage {
    uint256 nonce;
    bool locked;
    uint256 nonce4337;
    uint256[47] __gap;
}

// keccak256(abi.encode(uint256(keccak256("rewardy.dualentry.v3")) - 1))
//   & ~bytes32(uint256(0xff))
bytes32 private constant STORAGE_SLOT = 0x ...;
```

Impact

The impact materializes when Rewardy deploys V4 and begins re-delegating users:

1. **Nonce Reset / Collision:** If V4 stores its nonce at a different slot, the nonce effectively resets to 0.

2. **Reentrancy Guard Corruption:** If V4 moves `_locked` to a different slot or changes its type, the reentrancy guard reads stale data from V3's slot 1. A `true` residual could permanently lock sponsored execution; a missing guard could allow reentrancy.
3. **Event Counter Confusion:** `nonce4337` at slot 2 would be misread by V4, producing misleading events and potentially confusing off-chain indexers tracking 4337 operations.
4. **Cross-Contract Collision:** If a user accidentally re-delegates to an unrelated contract (e.g., through phishing), that contract's slot 0-2 data is interpreted as Rewardy state, with unpredictable consequences.

At 600K users with re-delegation as the planned upgrade path, the blast radius of a storage collision during a V3-to-V4 migration could affect the entire user base simultaneously.

Proof of Concept

Consider the following re-delegation scenario:

```
V3 state for user EOA 0xAlice :
  slot 0 (nonce)      = 42
  slot 1 (_locked)    = false
  slot 2 (nonce4337) = 17
```

Alice re-delegates to V4, which uses a different layout:

```
slot 0 = _locked      [reads 42 interpreted as true permanent lock]
slot 1 = nonce        [reads 0 [false as uint] nonce resets to 0]
slot 2 = feeAccum     [reads 17 V4 thinks 17 wei of fees accumulated]
```

Even if V4 keeps the same ordering but inserts a new field before `nonce`:

```
V4 layout :
  slot 0 = version    [reads 42 V4 thinks version is 42]
  slot 1 = nonce      [reads 0 nonce resets, enabling signature replay]
  slot 2 = _locked    [reads 17 true permanent reentrancy lock]
```

With ERC-7201, V3 and V4 storage would be at completely non-overlapping namespace-derived slots, preventing any collision regardless of layout changes.

Recommendation

Migrate to ERC-7201 namespaced storage before any re-delegation campaign.

1. Compute a unique namespace slot:

```
/// @custom:storage-location erc7201:rewardy.dualentry.v3
struct AccountStorage {
    uint256 nonce;
    bool locked;
    uint256 nonce4337;
    uint256[47] __gap;
}
```

```
// keccak256[abi.encode(uint256(keccak256("rewardy.dualentry.v3")) - 1)]
// & ~bytes32(uint256(0xff))
bytes32 private constant ACCOUNT_STORAGE_SLOT = 0x...; // compute at
// deploy time

function _getStorage() private pure returns (AccountStorage storage s) {
    bytes32 slot = ACCOUNT_STORAGE_SLOT;
    assembly {
        s.slot := slot
    }
}
```

2. Replace all direct slot references with `_getStorage().nonce`, `_getStorage().locked`, etc.
3. For V4, use a DIFFERENT namespace (`rewardy.dualentry.v4`), ensuring V4's storage lands at completely non-overlapping slots.
4. Consider implementing [ERC-7779](#) (designed specifically for EIP-7702 re-delegation storage conflict prevention) if the draft stabilizes before V4 ships.
5. If V3 is already deployed and a storage migration is infeasible, document a V4 deployment checklist that requires the V4 contract to explicitly read and clear V3's sequential slots 0-2 as part of a one-time migration call before switching to namespaced storage.

Team Response

Fixed.

[L-03] Stale ERC-4337 UserOperation Signatures Remain Valid Across EIP-7702 Re-delegation

Severity

Low Risk

Description

The `validateUserOp()` validates ERC-4337 UserOperations by recovering the signer from the EntryPoint-provided `userOpHash` and checking only that the recovered signer equals `address(this)`. In the intended EIP-7702 deployment model, `address(this)` is the delegated EOA address. That address remains stable even if the EOA later re-delegates to a different implementation.

This means a UserOperation signature produced while the EOA is delegated to the honest Rewardy implementation is not explicitly bound by the account to the active delegation implementation, implementation hash, or delegation epoch. If the same EOA is later re-delegated to attacker-controlled logic that preserves the same raw `recovered == address(this)` validation model, the stale UserOperation signature can still be accepted while execution semantics have changed.

The issue is valid as a defense-in-depth authorization-context gap, but it is Low Risk rather than High/Medium because exploitation requires a strong upstream compromise: the victim EOA must first be re-delegated to attacker-controlled code, or the attacker must otherwise control the replacement delegation logic. Once that precondition holds, the account's core trust boundary is already broken.

Location of Affected Code

File: [rwd.dual.entry.account.v3.sol#L269-L293](#)

```
function validateUserOp(
    PackedUserOperation calldata userOp,
    bytes32 userOpHash,
    uint256 missingAccountFunds
) external onlyEP returns (uint256 validationData) {
    if [_isPaused()] {
        return SIG_VALIDATION_FAILED;
    }

    if (userOp.sender != address(this)) {
        return SIG_VALIDATION_FAILED;
    }

    (address recovered, ECDSA.RecoverError err, ) = ECDSA.tryRecover(
        userOpHash,
        userOp.signature
    );

    // Signature is bound to the EOA identity, but not to a delegation
    // implementation, implementation hash, or delegation epoch.
    if (err != ECDSA.RecoverError.NoError || recovered != address(this)) {
        return SIG_VALIDATION_FAILED;
    }

    _payPrefund(missingAccountFunds);
    return SIG_VALIDATION_SUCCESS;
}
```

File: [rwd.dual.entry.account.v3.sol#L245-L267](#)

```
function execute(
    address target,
    uint256 value,
    bytes calldata data
) external payable onlyEP nonReentrant whenNotPaused {
    InternalCall[] memory calls = new InternalCall[](1);
    calls[0] = InternalCall({to: target, value: value, data: data});
    _executeBatch4337(calls);
}

function executeBatch(
    BatchCall[] calldata calls_
) external payable onlyEP nonReentrant whenNotPaused {
    InternalCall[] memory calls = new InternalCall[](calls_.length);
    for (uint256 i = 0; i < calls_.length; i++) {
        calls[i] = InternalCall({
            to: calls_[i].target,
```

```

        value: calls_[i].value,
        data: calls_[i].data
    });
}
_executeBatch4337(calls);
}

```

Impact

Pending ERC-4337 UserOperations can survive an EIP-7702 re-delegation boundary if the replacement implementation accepts the same signature model. A user may sign a UserOperation intending to call the Rewardy `execute` selector with benign arguments, but after re-delegation the same selector can be implemented with attacker-controlled semantics.

In the included PoC, a stale UserOperation originally prepared for a benign call is validated after re-delegation and then drains `4 ether` from the delegated EOA to the attacker.

The realistic impact is limited to transition windows where:

1. A valid UserOperation was signed before re-delegation and remains unexecuted.
2. The EOA is re-delegated to malicious or compromised logic before execution.
3. The replacement logic accepts the stale signature and reinterprets the signed call.

Proof of Concept

The repository already contains a focused Foundry reproduction in `test/RedelegationRisk.t.sol`.

Run:

```

forge test --match-test
    test_4337UserOpCanExecuteAgainstRedelegatedImplementation -vv

```

Relevant PoC excerpt from `test/RedelegationRisk.t.sol:340-397`:

```

function test_4337UserOpCanExecuteAgainstRedelegatedImplementation()
external {
    vm.deal(user, 4 ether);

    bytes memory innerData = abi.encodeWithSelector(
        BenignCallTarget.setValue.selector,
        21
    );
    bytes memory callData = abi.encodeWithSelector(
        RewardyDualEntryAccountV3.execute.selector,
        address(benignTarget),
        0,
        innerData
    );

    PackedUserOperation memory userOp;
    userOp.sender = user;
    userOp.nonce = 0;

```



```

userOp.initCode = "";
userOp.callData = callData;

bytes32 userOpHash = keccak256(
    abi.encode(
        userOp.sender,
        userOp.nonce,
        keccak256(userOp.initCode),
        keccak256(userOp.callData),
        block.chainid,
        address(entryPoint)
    )
);
userOp.signature = _signHash(userOpHash);

SwitchableDelegationHarness(payable(user)).setImplementation(
    address(malicious4337Impl)
);

uint256 validation = entryPoint.callValidateUserOp(
    RewardyDualEntryAccountV3(payable(user)),
    userOp,
    userOpHash
);
assertEq(validation, 0, "signature still validates after re-
delegation");

entryPoint.callExecute(
    RewardyDualEntryAccountV3(payable(user)),
    address(benignTarget),
    0,
    innerData
);

assertEq(
    benignTarget.calls(),
    0,
    "same execute selector now runs attacker semantics"
);
assertEq(attacker.balance, 4 ether, "reused UserOp drained ETH");
assertEq(user.balance, 0, "user balance should be drained");
}

```

Observed result:

```
[PASS] test_4337UserOpCanExecuteAgainstRedelegatedImplementation()
```

Recommendation

Do not treat the EOA address alone as sufficient authorization context across delegation changes. Harden both the account contract and off-chain signing pipeline so pending signatures are invali-

dated or rejected when the EOA's active delegation changes.

No check inside the old Rewardy implementation can force a fully malicious replacement implementation to behave safely after re-delegation. Therefore, the critical mitigation is to prevent stale signed payloads from being usable across Rewardy-controlled migrations and to invalidate them at the client/backend layer whenever delegation state changes.

Recommended fixes:

1. Use the EIP-7702-aware ERC-4337 flow for delegated EOAs instead of a plain empty-`initCode` `UserOperation` flow whenever possible, so the `EntryPoint/userOp` hash path includes the delegation context expected by the deployed `EntryPoint` version.
2. For Rewardy-controlled implementations, wrap the `EntryPoint` `userOpHash` in an account-owned digest before recovery, and include a delegation context field such as implementation address, implementation code hash, or a monotonic delegation epoch maintained by the wallet backend:

```
bytes32 digest = keccak256(
    abi.encode(
        keccak256("RewardyUserOp(bytes32 userOpHash,address
            implementation,uint256 delegationEpoch)"),
        userOpHash,
        expectedImplementation,
        expectedDelegationEpoch
    )
);

[address recovered, ECDSA.RecoverError err, ] = ECDSA.tryRecover(
    digest,
    userOp.signature
);
```

3. In the Rewardy client/backend, treat every re-delegation event as invalidating all pending `UserOperations` and cached signatures for that EOA.
4. Use short validity windows for `UserOperations`. If the chosen ERC-4337 flow supports `validUntil`/`validAfter`, return packed validation data that expires stale operations quickly.
5. During migration to a new Rewardy implementation, require fresh signatures after the new delegation is active and reject any operation signed before the migration boundary.

Team Response

Fixed.

[L-04] `whenNotPaused` Modifier is Not Reachable

Severity

Low Risk

Description

Since `validateUserOp()` already short-circuits with `SIG_VALIDATION_FAILED` when `paused`, the `EntryPoint` never proceeds to call `execute()` or `executeBatch()`.

```
handleOps
  validateUserOp
    if [_isPaused()] return SIG_VALIDATION_FAILED  stops here
  execute() / executeBatch()  never reached
  whenNotPaused  dead code when called via EntryPoint
```

When paused, the function returns `SIG_VALIDATION_FAILED` without calling `_payPrefund()`. The `EntryPoint` still expects the account to cover gas costs even for failed validations — hence `AA21 didn't pay prefund` error revert.

```
// @audit validation here
>> validationData = _callValidateUserOp(opIndex, op, opInfo,
missingAccountFunds);
    if [paymaster == address[0]] {
      // @audit payment checked here
      if (!_tryDecrementDeposit(sender, requiredPrefund)) {
>>         revert FailedOp[opIndex, "AA21 didn't pay prefund"];
      }
    }
  }
```

As such, the `whenNotPaused` modifiers on those functions are unreachable via the 4337 path.

Location of Affected Code

File: `rwd.dual.entry.account.v3.sol#L245-L253`

```
function execute(
    address target,
    uint256 value,
    bytes calldata data
) external payable onlyEP nonReentrant whenNotPaused {}
```

File: `rwd.dual.entry.account.v3.sol#L255-L257`

```
function executeBatch(
    BatchCall[] calldata calls_
) external payable onlyEP nonReentrant whenNotPaused {}
```

Recommendation

Consider removing the modifiers.

Team Response

Fixed.

[I-01] `validateUserOp()` Missing Explicit `initCode` Rejection

Severity

Informational Risk

Description

The repository documentation states that `validateUserOp()` rejects non-empty `initCode`, but the on-chain implementation does not enforce that check.

In the current implementation, `validateUserOp()` checks pause state, sender equality, and signature recovery, then returns success without branching on `userOp.initCode.length`.

Location of Affected Code

File: [rwd.dual.entry.account.v3.sol#L269-L293](#)

```
function validateUserOp(
    PackedUserOperation calldata userOp,
    bytes32 userOpHash,
    uint256 missingAccountFunds
) external onlyEP returns (uint256 validationData) {
    if (_isPaused()) {
        return SIG_VALIDATION_FAILED;
    }

    if (userOp.sender != address(this)) {
        return SIG_VALIDATION_FAILED;
    }

    (address recovered, ECDSA.RecoverError err, ) = ECDSA.tryRecover(
        userOpHash,
        userOp.signature
    );

    if (err != ECDSA.RecoverError.NoError || recovered != address(this)) {
        return SIG_VALIDATION_FAILED;
    }

    _payPrefund(missingAccountFunds);
    return SIG_VALIDATION_SUCCESS;
}
```

Impact

Spec Mismatch

Recommendation

Add the explicit rejection branch documented by the project:

```
if [userOp.initCode.length != 0] {  
    return SIG_VALIDATION_FAILED;  
}
```

Team Response

Fixed.

[I-02] Unused Interface Declarations

Severity

Informational Risk

Description

The account contract declares interface elements that are not consumed by runtime logic:

1. `IRewardyPauseBeacon` is declared but never used as a type or call target.
2. `IEntryPoint.getNonce(address,uint192)` is declared but never called.

Location of Affected Code

File: [rwd.dual.entry.account.v3.sol#L8-L17](#)

```
interface IEntryPoint {  
    function getNonce(  
        address sender,  
        uint192 key  
    ) external view returns [uint256]; // declared but not called  
}  
  
interface IRewardyPauseBeacon {  
    function isPaused() external view returns [bool]; // interface  
        declared but unused  
}
```

Impact

Unused code

Recommendation

Keep only declarations that are actively used in logic.

- Remove `IRewardyPauseBeacon` if pause beacon access remains low-level assembly/staticcall-based.

- Remove `IEntropyPoint.getNonce(...)` from the interface if nonce reads are intentionally delegated to `EntropyPoint` internals and not queried by this account.

Team Response

Fixed.

our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Thank you!

